

CEO AI Thailand — MOAT Architecture v1

23 ส.ค. 2569 · เจ้าของ freeze Strategic North Star — เอกสารนี้คือ สถาปัตยกรรม ไม่ใช่คอนเทนต์
🔒 ไม่มี migration และไม่แตะฐานข้อมูลในเอกสารนี้ (Gate B ยังไม่ปิด) — ทั้งหมดเป็น สัญญาของโครงสร้าง
ที่ schema จริงต้องเคารพเมื่อถึงเวลาสร้าง ⇒ ตอนสร้างตารางจริงให้ derive จากที่นี่ ไม่ใช่คิดใหม่

0. ประโยคเดียวที่หึงเอกสารนี้รับใช้

CEO AI Thailand = AI Business Validation-to-Scale Operating System สำหรับคนไทย
ช่วยคนไทยเปลี่ยนไอเดียให้เป็นธุรกิจที่มีลูกค้า มีหลักฐาน มีระบบ และขยายได้
โดย AI แนะนำ Next Best Business Action จาก Business Genome · Experiment Evidence · Learning Loop

⚠️ นี่คือป้ายภายใน — พาดหัวที่คนเจอครั้งแรกยังคงเป็น ปัญหา: "อย่าเพิ่งสร้างธุรกิจ จนกว่าจะรู้ว่าใครจะซื้อ"
(เหตุผล: ไม่มีใครค้นหาชื่อหมวดหมู่ · competitiveStrategy.CATEGORY)

1. ห่วงโซ่ที่เปลี่ยน POD ให้เป็น MOAT (ห้ามข้ามขั้น)

POD → Workflow → Proprietary Decision Rules → Structured Business Data
→ Outcome Learning → Benchmark → Data Network Effect → MOAT

● อยู่ในโค้ดแล้ว: `competitiveStrategy.NORTH_STAR.chain` (เทสต์บังคับว่าเริ่มที่ POD จบที่ MOAT)

ขั้น	POD	สิ่งที่คู่แข่ง copy ยากขึ้น
1	Validation Before Spending	Workflow
2	Next Best Business Action	Decision Rules
3	Idea to Scale Journey	Structured proprietary data
4	Evidence-Based Business Building	Learning dataset
5	Business Systemization	Network/Data moat

2. Business Genome — หัวใจของ moat

● สร้างแล้ว: `src/lib/businessGenome.ts` (19 เทสต์)

8 กิ่ง — Business · Customer · Problem · Offer · Acquisition · Experiment · Economics · Scale
ทุกกิ่งผูกกับขั้นของ Customer Journey และ เทสต์บังคับว่าขั้น 1-7 ต้องมีกิ่งรองรับครบ
⇒ ถ้าสัญญาอะไรใน Journey แล้วไม่มีที่เก็บข้อมูล = แดงทันที

● สิ่งที่ไม่ใช่ จีโนม (เขียนไว้กันคนเข้าใจผิด)

ประวัติแชต · prompt ที่เคยใช้ · ไฟล์ที่อัปโหลด · บันทึกการใช้งานรายวัน

ของพวกนี้ ลอกได้ทันทีที่เปลี่ยนผู้ให้บริการ LLM และไม่ได้บอกอะไรเกี่ยวกับ ธุรกิจ moat คือข้อมูลที่มีโครงสร้าง · สะสมข้ามเวลา · ผูกกับผลลัพธ์จริง

stuckBranch() — ตอบว่าธุรกิจติดตรงไหน โดยไม่ต้องถามซ้ำ

กิ่งแรกที่ยังไม่ครบ = จุดที่ติดจริง ไม่ใช่จุดที่เจ้าของอยากทำ — นี่คือวัตถุดิบของ Next Best Action

3. Evidence Graph — Business Memory

Claim → Hypothesis → Experiment → Evidence(observed) → Outcome → Learning → Confidence

● สร้างแล้ว: businessGenome.EvidenceNode + confidenceOf()

กรอกอะไร	ได้ความมั่นใจระดับ
สมมติฐาน + วิธี	research
+ สิ่งที่เกิดขึ้นได้	observed
+ ผลจริง และ บทเรียน	validated

! ผลจริงอย่างเดียวไม่พอ — ต้องมีบทเรียนด้วยถึงจะเป็น validated (เทสต์บังคับ)

เพราะผลที่ไม่ถูกรูปเป็นบทเรียน ไม่ไหลกลับเข้าจีโนม ⇒ ไม่สะสมเป็น moat

ตัวอย่างจริงที่เจ้าของเขียน (เข้าโครงนี้พอดี):

สมมติฐาน ร้านอาหารอยากลด food cost → สัมภาษณ์ 20 ร้าน → observed 14 ร้านมี cost variance → outcome 3 ร้านซื้อ → learning pain จริง แต่ WTP ขึ้นกับขนาดร้าน ⇒ รอบหน้า AI ไม่ได้เริ่มจากศูนย์

4. Competition Memory + ด้าน "ต่างจริงไหม"

● สร้างแล้ว: src/lib/competitorMemory.ts

```
Proposed POD → ตลาดพุดกันแล้วหรือยัง?
  YES → POP
  NO → ลอกง่ายไหม?
    YES → Weak POD
    NO → มีหลักฐานไหม?
      YES → Strategic POD
      NO → Hypothesis POD
```

KNOWN_MARKET เก็บ 5 ราย — รวม "Excel / จดมือ / ไม่ทำอะไรเลย" ซึ่งเป็นคู่แข่งที่ชนะบ่อยที่สุดและมักถูกลืม
ทุกรายต้องมี whiteSpace (เทสต์บังคับ) — ความจำที่ไม่บอกช่องว่าง = จำไว้แล้วไม่ได้ใช้

⚠ ความจำที่ไม่อัปเดต = ความมั่นใจปลอม — เจอคู่แข่งใหม่ต้องเพิ่มเข้ารายการ

5. VRIO Engine — ให้คะแนนจริง

🟢 สร้างแล้ว: `competitiveStrategy.VRIO_ASSETS + vrioVerdict()`

Asset	V	R	I	O	ผล
AI Content	5	1	1	5	POP
MIT Workflow	5	3	2	4	Temporary POD
Decision Engine	5	4	3	4	Strong POD
Business Genome	5	4	4	4	Emerging Moat
Thai Outcome Dataset	5	5	5	4	Potential Moat
Benchmark Network	5	5	5	5	Strong Moat

- 🔴 R และ I เป็นตัวชี้ขาด ไม่ใช่ V — V=5 แต่ R/I ต่ำ ได้ POP เสมอ (เทสต์บังคับ)
- 🔴 O ต่ำต้องไม่ถูกปิดขึ้น — ของที่ยังไม่ได้สร้างต้องดูเหมือนยังไม่ได้สร้าง
- ⚠ คะแนนนี้เป็น การประเมินของเรา ไม่ใช่ผลวัด — ห้ามยกไปอ้างกับลูกค้าว่าเป็นข้อเท็จจริง

6. 🗺 Map ลงระบบจริง

6.1 Database Schema — spec เท่านั้น ยังไม่สร้าง

กิ้งจิงโนม	ตารางที่ควรเป็น	หมายเหตุ
business · customer · problem · offer	<code>business_genome</code> (JSONB ต่อ workspace)	เริ่มเป็น JSONB ก่อน แดกเป็นตารางเมื่อ query pattern ชัด
experiment	<code>experiments</code>	1 แถว = 1 สมมติฐาน · ต้องมี FK ไป workspace
evidence	<code>evidence_nodes</code>	ผูกกับ experiment · เก็บ observed/outcome/learning แยกช่อง
acquisition · economics	ต่อยอดจาก <code>landing_funnel</code> + <code>finance</code> ที่มีอยู่	อย่าสร้างซ้ำของที่มีแล้ว
scale	ต่อยอดจาก <code>processRegister</code> ที่มีอยู่	มี export CSV/JSON แล้ว

🔒 เงื่อนไขก่อนแตะ schema (เจ้าของกำหนด · ห้ามข้าม):

Gate B PASS/FROZEN → Phase 1 Acceptance #2 → Freeze Phase 1 baseline
→ เพิ่ม VRIO/POP/POD Strategy Layer เป็น controlled next iteration
→ เปิด regression gates เฉพาะส่วนที่ได้รับผลกระทบ

● ผู้ช่วยมองไม่เห็น Gate B / Phase 1 Acceptance ในรีโปนี้ (grep แล้วไม่พบ)
⇒ ปฏิบัติตามคำสั่งโดยไม่ได้ตรวจสอบเอง — ตรวจไม่ได้ ไม่ใช่ตรวจแล้วผ่าน

6.2 Agent Architecture

Agent	ต้องเรียกอะไรก่อนทำงาน
StrategyAgent	<code>reviewCampaign()</code> — ไม่ผ่านด่าน = ไม่ออกแคมเปญ
ContentAgent	<code>positioningBlock()</code> ใน prompt + <code>CONTENT_DNA</code> ลำดับตายตัว
ResearchAgent	<code>assessDifferentiation()</code> ก่อนบอกอะไร "ต่าง"
AnalyticsAgent	<code>MEASUREMENT_SAFETY_GUARDS</code> — ต่ำกว่าเกณฑ์ = รายงานจำนวนคน ไม่ใช่ %
ทุก Agent	<code>brandBriefBlock()</code> ซึ่งพา strategy + positioning ติดไปด้วยอัตโนมัติ

6.3 SKILL.md

`.claude/skills/moat-architecture/SKILL.md` — เปิดอ่านก่อน ออกแบบฟีเจอร์ใหม่ทุกครั้ง
คำถามเดียวที่ skill นี้บังคับถาม: "ฟีเจอร์นี้เพิ่ม moat หรือเพิ่มแค่จำนวนฟีเจอร์"

6.4 Marketing OS = Moat Generator

Audience → Problem → POD → Campaign Hypothesis → Content → Experiment
→ Tracking → Outcome → Learning → Business Genome → Better Campaign

⇒ ทุกแคมเปญต้อง เพิ่ม intelligence ให้ระบบ ไม่ใช่แค่ผลิตคอนเทนต์เพิ่ม

6.5 Learning Engine

Hypothesis → Campaign → Experiment → Customer Response → Outcome
→ Learning → Business Genome → Decision Rule → Better Next Campaign

● ยังสร้างไม่ได้ — `moatReadiness()` บอกว่าต้องมีผู้ใช้จริง ≥ 1 (Experiment Memory) และ ≥ 30 (Learning Loop)

7. ทำได้ตอนนี้ / ต้องรอ — ตัดสินด้วยตัวเลข ไม่ใช่ความรู้สึก

`moatReadiness(activeUsers)` เป็นตัวตัดสิน · วันนี้ผู้ใช้งานนอกที่ใช้จริง = 0

	สิ่งที่ทำ	สถานะ
	Business Genome (ontology)	 สร้างแล้ววันนี้
	Thai Business Playbook (decision rules)	 โดกรพร้อม — ยังต้องเขียนกฎ

	สิ่งที่ทำ	สถานะ
🕒	Experiment Memory	ผู้ใช้ ≥ 1
🕒	Decision Engine (ที่เรียนรู้จากผลลัพธ์จริง)	ผู้ใช้ ≥ 5
🕒	Benchmark Network	ผู้ใช้ ≥ 30 + ความยินยอม PDPA
🕒	Learning Loop ปิดวงจร	ผู้ใช้ ≥ 30

⚠️ ข้อควรรู้เรื่อง Next Best Business Action: กฎ เขียนได้ตั้งแต่วันนี้ (เป็นส่วนของ Thai Playbook) แต่ เอนจินที่เก่งขึ้นจากผลลัพธ์จริง ต้องรอผู้ใช้ — สองอย่างนี้คนละสิ่ง อย่าสับสน

8. ตัวอย่างกฎที่เขียนได้ตั้งแต่วันนี้ (Thai Business Playbook)

```
IF problem_validation < threshold
THEN block paid_acquisition_recommendation
  NEXT_ACTION = customer_interview

IF repeat_sales = TRUE AND process_repeatability = LOW
THEN recommend SOP/process_design BEFORE aggressive_scaling
```

⚠️ กฎพวกนี้ ยังไม่เคยถูกทดสอบกับธุรกิจจริง ⇒ สถานะ hypothesis ห้ามนำเสนอว่าเป็น "ระบบที่รู้ว่าควรทำอะไร" จนกว่าจะมีเคสจริงมาหักล้าง

9. เส้นที่ต้องไม่ข้าม

🚫 ห้าม	เพราะ
ประกาศว่ามี moat ที่ยั่งยืนแล้ว	R/I ที่แข็งเกิดหลังมีข้อมูลผลลัพธ์จริง · วันนี้ 4 บัญชี · จ่ายจริง 0 ราย
อ้าง "data network effect"	ยังไม่มีผู้ใช้ให้เกิด network
เตะ schema/migration ก่อน Gate B ปิด	security evidence ที่กำลังเก็บจะต้อง regression ใหม่
สร้าง Decision Engine ที่ "เรียนรู้" ตอนนี้	กฎที่ไม่เคยถูกทดสอบ = ความมั่นใจปลอม
เอา North Star ขึ้นเป็นพาดหัว	ไม่มีใครค้นหาชื่อหมวดหมู่

สรุปหนึ่งประโยค

ฟีเจอร์ที่ไม่ทำให้จีโนมสมบูรณ์ขึ้น ไม่ทำให้กฎการตัดสินใจแม่นยำขึ้น หรือไม่ทำให้ผลลัพธ์ไหลกลับเข้าระบบ = เพิ่มจำนวนฟีเจอร์ ไม่ได้เพิ่ม moat